Zephyr Project

IT-Schulungen.com
Technology & Transformation

Zeitverhalten in Zephyr

Timeouts, Periodik und Messgrenzen

2.07

Drei Timeout-Semantiken

01	`K_NO_WAIT`	nicht warten · sofort zurückkehren
02	`K_MSEC() / K_SECONDS()`	endlich warten · Einheit im Code sichtbar machen
03	`K_FOREVER`	unbegrenzt warten · nur mit bewusstem Freigabepfad

Timeouts sind Teil der Fehlerstrategie: sofort, begrenzt oder unbegrenzt ist eine Entwurfsentscheidung.

Relative Pause: einfach, aber driftanfällig

RELATIVE PERIODE

```
while (1) {
    do_work();
    k_sleep(K_MSEC(100));
}
```

01 Effektive Periode

Die Arbeitszeit kommt zur Schlafzeit hinzu.
Periode = Laufzeit von do_work() + 100 ms.

02 Drift

Schwankende Arbeitszeit verschiebt jeden Folgestart.
Für unkritische Wiederholungen kann das trotzdem genügen.

k_sleep() beschreibt eine relative Pause – noch keinen stabilen periodischen Startzeitpunkt.

Periodik mit Timepoint

STABILE PERIODE

```
while (1) {
    sys_timepoint_t deadline =
sys_timepoint_calc(K_MSEC(100));
    do_work();
    k_sleep(sys_timepoint_timeout(deadline));
}
```

01 Zielzeit

Deadline wird am Periodenanfang berechnet.

02 Restzeit

Geschlafen wird nur bis zum Zielzeitpunkt.

03 Überziehung

Dauert die Arbeit zu lange, bleibt keine positive Wartezeit.

Vier Begriffe sauber trennen

MESSMODELL

Beobachtung ≠ automatisch Echtzeitnachweis

Auflösung	kleinster darstellbarer oder messbarer Zeitschritt
Genauigkeit	Nähe des Messwerts zum tatsächlichen Zeitpunkt
Jitter	Schwankung von Start- oder Abschlusszeitpunkten
Deadline	spätester Zeitpunkt, zu dem ein Ergebnis vorliegen muss

Eine periodische Ausgabe beweist weder kleine Jitterwerte noch eingehaltene Deadlines.

Messwerte richtig einordnen

`Terminalausgabe`	Logging und Ausgabe verändern die beobachtete Laufzeit
`Hostlast`	andere Prozesse beeinflussen Planung und Messwerte
`native_sim`	gut für funktionale Tests, kein Hardware-Echtzeitnachweis
`Überziehung`	verpasste Periode erkennen und bewusst behandeln
`Zielhardware`	Timing-Anforderungen unter realer Last und Peripherie prüfen

Messumgebung, Last, Auflösung und Beobachtungseffekt gehören immer zum Ergebnis.

Kurzcheck: Zeitverhalten

TIMEOUT

Semantik wählen

K_NO_WAIT: sofort

K_MSEC(): begrenzt

K_FOREVER: unbegrenzt

Fehlerpfad mitdenken

PERIODIK

Startzeit stabilisieren

relative Pause erzeugt Drift

Timepoint definiert Ziel

Überziehung sichtbar machen

Uptime zur Beobachtung nutzen

AUSSAGE

Messung begrenzen

Hostlast dokumentieren

native_sim einordnen

Zielhardware verifizieren

Transferfrage: Welche Aussage dürft ihr aus einer stabilen native_sim-Ausgabe nicht ableiten?